# FOL: Bridging Object-Oriented and Functional Programming via the Metaobject Protocol

Frank Adrian
Ancar Technology
Olathe, KS, USA
frank.adrian314159@gmail.com

## Abstract

Clojure [16] omits CLOS; Hickey [17] argues that mutable objects are a complexity hazard incompatible with immutability rather than the power boost object-orientation promises. In FOL (Functional Object Lisp), we use the Metaobject Protocol (MOP) to back instances with persistent structures, preserving CLOS's open dispatch model while enforcing immutable reference semantics. FOL contributes a persistent metaclass with hybrid storage and lazy migration, and a specificity-ordered predicate dispatch model.

FOL's slot reads incur 1.4–2.2× overhead; slot updates cost 40–250× in isolation. At idiomatic workload scales, FOL is 5–44× slower than mutable CLOS; for direct translation of fine-grained mutable algorithms (e.g., AST rewriting), overhead reaches 202×. However, at extreme scale, persistent state maps avoid the hash-table fragmentation that degrades the mutable baseline, yielding a 3.8× runtime advantage in logic simulation. Predicate dispatch uses a level-based ordering sorted at each method-definition in $O(ND \log N)$ ($D$ = hierarchy depth), avoiding the pairwise theorem-prover overhead of subsumption systems; the trade-off is an $O(N)$ per-call linear scan (vs. $O(1)$ cached dispatch), with closed `defn` dispatch indistinguishable from hand-written `cond` cascades (≈2.0×).

## CCS Concepts

• **Software and its engineering → Constraints**; **Classes and objects**; **Functional languages**; **Object oriented languages**; *Extensible languages.*

## Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP

## 1 Introduction

CLOS is extensible, but mutable objects create three problems for speculative workloads, evolving schemas, or value-dispatched guards:

**P1 — Costly structural sharing.** CLOS lacks native structural sharing; speculative workloads must copy objects in $O(N)$ in the number of slots for every snapshot. This makes fine-grained snapshotting prohibitively expensive compared to persistent alternatives (FSet [5], Sycamore [9]), which nonetheless lack CLOS integration. FOL's isolated slot updates cost 40–250×, but $O(\log n)$ structural sharing bounds per-update cost; at sufficient scale, persistent instance state maps avoid the hash-table fragmentation that degrades mutable baselines (Section 5.4).

**P2 — Schema evolution labor.** Renaming slots requires manual `update-instance-for-redefined-class` methods. FOL's `:alias` annotation recovers renamed slots at first access, composing multi-hop chains in a single pass (Section 5.1.2).

**P3 — Inexpressive predicate dispatch.** CLOS dispatch is type-limited; value-level conditions require manual guards. `filtered-functions` [24] extends this via subsumption but requires expensive pairwise logical-implication checks over Boolean predicates. FOL's syntactic ordering is decidable at load time in $O(ND \log N)$; per-call overhead approaches manual `cond` cascades (≈2.0×; Section 5.1.1).

FOL uses the MOP [19] to back instances with persistent structures and adds specificity-ordered predicate dispatch. Slot mutation becomes `assoc`, while other CLOS interfaces remain (Table 1). The contributions compose: Listing 4 gates on values while reading persistent state, while Listing 3 enables $O(1)$ update discard without copying.[1]

Contributions:

(1) A **persistent metaclass system** (Sec. 2) with hybrid native/trie storage and lazy alias-based migration (Sec. 4.2).
(2) A **specificity-ordered predicate dispatch** (Sec. 4) using load-time decidable leveling without theorem provers, integrated into CLOS.

## 2 Architecture

Slot storage uses a hybrid layout: each persistent object has $T = 32$ native positions, 6 reserved for system overhead (trie pointer, schema version, metadata), leaving 26 for user slots. Most classes (≤26 slots) fit this layout [20]; we confirmed this via a survey of 295 Quicklisp definitions where 97% of classes fell within this limit. Redefinitions are lazy: renames resolve via alias maps; new slots acquire initforms at first mutation. Redefinitions exceeding 26 slots trigger trie conversion.

`persistent-class` ensures immutability. By dropping `change-class` and `(setf slot-value)`, FOL replaces state-machine identity with functional projection, borrowing CLOS syntax and MOP mechanics. `assoc` returns a fresh instance sharing nodes via path copying.

### 2.1 Collections and Runtime

FOL implements persistent collections via 32-way tries [3, 23]. FOL transpiles to Common Lisp via a two-stage pipeline: (1) the *Pattern Reader* parses extended syntax, and (2) the *Macro Expander* emits standard CL. This leverages the host compiler's optimizations while maintaining FOL's semantics.

[1] FOL addresses the Expression Problem [27]: new functions and clauses can be added over existing classes without modifying original code.

**Table 1: MOP Protocol Adaptation**

| Protocol | Status | Rationale |
|---|---|---|
| `slot-value-using-class` | Redirected | Reads from base object or overflow trie |
| `(setf slot-value-using-class)` | Guarded | Blocks post-init writes |
| `slot-boundp-using-class` | Redirected | Checks object slot membership |
| `slot-makunbound-using-class` | Blocked | Immutability invariant — use `dissoc` to unbind slots |
| `validate-superclass` | Extended | Cross-metaclass mixing |
| `initialize-instance` | Extended | Populates base object and overflow trie from initargs; stamps `%schema-version` (an integer matching the class's current version counter) at birth, enabling `update-instance-for-redefined-class` to detect stale instances |
| `reinitialize-instance` | Extended (`:before` on metaclass) | Snapshots current alias-map, overflow indices, and version counter into a `class-version` struct before each redefinition; advances the version counter |
| `compute-slots` | Specialized | Implements hybrid native/trie layout |
| `update-instance-for-redefined-class` | Implemented (`:after`) | Multi-hop chain replay: composes alias-maps from version chain back to birth version, recovering renamed values (Sec. 4.2) |
| `change-class` | Omitted | Changing a live object's class violates immutability; callers construct a new instance of the target class instead. Internal SBCL redefinition triggers are handled by constructing a fresh instance from the old state during the next access. |

*Method combination uses standard CLOS execution semantics (`:before`/`:after`/`:around`). $\prec_{disp}$ replaces the class precedence list; as a total order, it preserves CLOS's most/least-specific-first guarantees; definition index serves as the final tiebreaker that ensures totality.*

## 2.2 Space Complexity

Identity types hold current values; unreferenced snapshots are GC-eligible. Each `assoc` copies one root-to-leaf path ($O(\log_{32} N)$ nodes), bounding allocation by depth rather than size. SBCL nursery reclamation keeps GC overhead at ≈1.5% in the LSim benchmark (profiled at the 32x32x32-30000 configuration via `sb-sprof`).

# 3 Features

FOL adopts Clojure's literal syntax and sequence APIs. To ensure interoperability while maintaining functional semantics, FOL shadows `cl:assoc` with a generic function; within the `fol` package, `assoc` dispatches to persistent structures, while remaining accessible to standard lists via a fallback method. **Transducers**, **Lazy sequences**, **Atoms**, **Refs**, and **Agents** provide Clojure-parity in a Common Lisp setting; these are direct ports of Clojure's semantics and are verified via Clojure's original test suites. **Transient builders** reduce bulk update overhead. **Macros** follow Clojure-style expansion. Generic functions dispatch by arity, then specificity (Section 4). **Closed dispatch** (`defn`) emits a static `cond` cascade; **open dispatch** (`defmethod`) supports CLOS method combinations and extensibility. Table 2 summarises the criteria. The two forms are mutually exclusive per function name: `defn` emits `defun` and `defmethod` emits `defgeneric`.

# 4 Predicate Dispatch

Figure 1 formalizes FOL's multi-pattern dispatch syntax.

**Table 2: Choosing `defn` vs. `defmethod`**

| Requirement | `defn` | `defmethod` |
|---|---|---|
| Downstream code may add clauses | × | ✓ |
| `call-next-method` needed | × | ✓ |
| `:before`/`:after`/`:around` qualifiers | × | ✓ |
| Minimum dispatch overhead (static cond) | ✓ | × |

Note that *s-clause* with embedded map destructuring is the idiomatic equivalent of *m-clause*, which serves as a shorthand.

## 4.1 Dispatch Ordering Rules

FOL's predicate specializers (`var (fn args...)`) evaluate at dispatch time; predicates are more specific than types. Specificity categories (Table 3) establish a partial order; incomparable patterns at the same level resolve by explicit preference registry (see Section 4) or definition index. $\prec_{\text{disp}}$ conservatively approximates semantic specificity: Level 3 > 2 > 1 > 0. Level 2 uses semantic subclassing (Spec-Type); multi-parameter patterns apply Spec-Prefix and Spec-Tail position by position. Level 1 sets fixed-vs-rest precedence (Spec-Fixed). Consider:

```
(defn classify
  ([x <integer>]  :any-integer)   ; catch all
  ([x (even?)]    :even))         ; special case
```

FOL reorders this to `even?` first, so even integers never reach `:any-integer`. User-defined predicates must be pure and total

```
defn       ::=  (defn name (s-clause | m-clause)) | (fn (s-clause | m-clause))
defg/m     ::=  (defgeneric name...) | (defmethod name qualifier? (s-clause | m-clause))
s-clause   ::=  [param* ] body+
m-clause   ::=  {(symbol param | :keys [(symbol | (sym param))* ])* } body+
param      ::=  symbol | (symbol type) | (symbol (fn arg*)) | destr
destr      ::=  [param* ] | {:keys [(symbol | (sym param))* ]}
type       ::=  <type-name>
qualifier  ::=  :before | :after | :around
```

**Figure 1: FOL Multi-Pattern Dispatch Grammar (compressed)**

over all possible argument types; the dispatch scan provides no condition-handler boundary, so a condition signalled inside a predicate propagates directly to the caller. For closed `defn` groupings, syntactically decidable shadowed clauses trigger a compilation warning. Definition-order reliance creates modularity hazards; we use explicit mechanisms like `prefer-method` to mitigate this issue.

Method qualifiers execute in CLOS-standard order (e.g., most-specific-first `:before`), but $\prec_{\text{disp}}$ replaces the CPL. This deliberately diverges: predicates execute before types, prioritizing behavioral constraints over structural inheritance.

**Table 3: Predicate Specificity Categories**

| Level | Category | Example |
|---|---|---|
| 3 | Predicates | `(x (even?))` |
| 2 | Types | `(x <integer>)` |
| 1 | Fixed-arity (all-wildcard params) | `[x y]` |
| 0 | Wildcard | `x` |

Conjunction maps to its components' maximum level, providing a conservative upper bound *across levels only*: two Level-3 patterns—e.g., `(and (even?) (prime?))` and `(even?)`—both sit at Level 3 and are incomparable under $\prec$; semantic containment between same-level conjunctions is not captured syntactically and falls to the preference registry or *idx*. Disjunction `(or p q)` merges distinct domains, making it semantically less specific than either conjunct; the uniform level calculus cannot express this relationship, so `or` patterns are treated as atomic Level-3 predicates and ordered by the tiebreaker when paired against other Level-3 patterns. Negation is excluded: `(not p)`'s ordering requires knowing the complement domain, which is undecidable in the general case; `not` patterns likewise fall to the tiebreaker.

**Known deficiency — type-annotated conjunctions.** `(and (<integer>) (prime?))` receives Level 3 ($\max(2,3) = 3$), the same level as the bare predicate `(prime?)` alone, even though the conjunction is strictly more constrained. The syntactic calculus cannot determine this semantic subsumption natively. The only mitigation currently is definition order, which introduces a modularity hazard. However, even without a theorem prover, simple structural subsumption (if pattern $A$ structurally contains type $T$ and predicate $P$, and pattern $B$ only contains predicate $P$, then $A \prec B$) could be trivially implemented at load-time to solve this cleanly. A future composite level (e.g., `(x <integer> (prime?))`) could enforce this structural subsumption statically.

*4.1.1 Formal Specificity Ordering.* We formalize the specificity relation $\prec$ using the inference rules summarized in Table 4. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern $p$. **Syntactic form classes**: wildcard (*wc*), type annotation (*ty*, written `<name>`), predicate (*pred*, written `(fn args...)`), sequential (*seq*, written `[...]`), and map (*map*, written `{...}`; sorted canonically via `string<` on symbol-names, keywords preceding symbols and strings). $p \sim q$ iff $\mathcal{L}(p) = \mathcal{L}(q)$ and $p, q$ belong to the same form class. For compound patterns, $\mathcal{L}([p_1, \ldots, p_n]) = \max_i \mathcal{L}(p_i)$ and $\mathcal{L}(\{k_1\ p_1, \ldots\}) = \max_i \mathcal{L}(p_i)$: the level is the max element level. Spec-Level applies to compound patterns globally: $[x\ (even?)]$ (L=3) fires before $[x$ <integer>$]$ (L=2). Positional rules (Spec-Prefix, Spec-Tail) apply only when compound patterns share the max level ($p \sim q$); Spec-Level takes precedence otherwise. **Convention:** $p_1 \prec p_2$ means $p_1$ *fires before* $p_2$—i.e., $p_1$ is more specific. Higher levels are more specific, so Spec-Level concludes $p_1 \prec p_2$ when $\mathcal{L}(p_1) > \mathcal{L}(p_2)$.

**Table 4: Specificity Inference Rules**

| Rule | Formalization |
|---|---|
| Spec-Level | $\dfrac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$ |
| Spec-Type | $\dfrac{C_1 \sqsubset C_2}{(x\ C_1) \prec (x\ C_2)}$ |
| Spec-Prefix | $\dfrac{p_1 \prec q_1 \qquad \mathcal{L}([p_1, \ldots]) = \mathcal{L}([q_1, \ldots])}{[p_1, \ldots] \prec [q_1, \ldots]}$ |
| Spec-Tail | $\dfrac{p_1 \sim q_1 \qquad [p_2, \ldots] \prec [q_2, \ldots]}{[p_1, \ldots, p_n] \prec [q_1, \ldots, q_n]}$ |
| Spec-Fixed | $\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \ldots n\}}{[p_1, \ldots, p_n] \prec [q_1, \ldots, q_n\ \&\ r]}$ |
| Spec-Map-Prefix | $\dfrac{p_1 \prec q_1 \qquad \mathcal{L}(\{k_1\ p_1, \ldots\}) = \mathcal{L}(\{k_1\ q_1, \ldots\})}{\{k_1\ p_1, \ldots\} \prec \{k_1\ q_1, \ldots\}}$ |
| Spec-Map-Tail | $\dfrac{p_1 \sim q_1 \qquad \{k_2 \ldots\} \prec \{k_2 \ldots\}}{\{k_1\ p_1, k_2\ p_2, \ldots\} \prec \{k_1\ q_1, k_2\ q_2, \ldots\}}$ |
| Spec-Keys | $\dfrac{\text{keys}(q) \subsetneq \text{keys}(p)}{p \prec q}$ |

Rule application priority: Spec-Level takes precedence over all positional rules; within the same level, Spec-Type refines ordering for Level-2 atomic patterns via the class hierarchy DAG ($C_1 \sqsubset C_2$ = transitive subclass relation). When two Level-2 patterns specialize

on sibling classes (neither a subclass of the other), Spec-Type does not apply and $\prec_{\text{disp}}$ falls back to the preference registry or *idx*; this is a known modularity hazard analogous to the Level-3 case. Spec-Prefix applies when first elements are strictly ordered ($p_1 \prec q_1$); Spec-Tail recurses on the remainder when heads are level-equal *and share the same syntactic form* ($p_1 \sim q_1$); the two rules are disjoint. Multi-parameter patterns apply the rules lexicographically position by position, after Spec-Level has partitioned patterns by max level.

For `defmethod`, the dispatch sort $\prec_{\text{disp}}$ re-sorts at each method-definition event, including REPL re-evaluation; for `defn`, the ordering is fixed at compile time. Let $idx(p)$ denote the definition index of the method clause containing $p$. $\prec_{\text{disp}}$ is defined by the comparator: (1) $\mathcal{L}(p)$ descending; (2) if same level and $p_1 \prec p_2$, then $p_1$ sorts first; (3) if same level and incomparable under $\prec$, the preference registry takes precedence; if no preference exists, $idx(p)$ descending breaks the tie. The partial order $\prec$ is applied strictly within levels; *idx* serves as the final tiebreaker.

Exact-arity patterns of different lengths are incomparable; dispatch isolates the matching arity group first. For $n$ arguments, this group contains all length-$n$ fixed-arity patterns and length-$\leq n$ rest-accepting patterns. $\prec_{\text{disp}}$ is total over this group (Lemma 1). Spec-Fixed resolves the overlap by ordering the fixed-arity pattern as more specific. Preference-based and definition-order tie-breaking aligns with Lisp's interactive workflow but introduces the modularity hazards discussed in Section **??**.

Lemma 4.1 ($\prec$ is a strict partial order; $\prec_{\text{disp}}$ is a strict total order). *The syntactic specificity relation $\prec$ defined by Table 4 is a strict partial order on arity-group patterns. The dispatch comparator $\prec_{disp}$—which extends $\prec$ with the preference registry and the definition-index tiebreaker idx—is a strict total order on arity-group patterns.*

Proof. *$\prec$ is a strict partial order.* Irreflexivity: every rule requires a strict inequality (level, element position, key set, or subclass) that is false when $p_1 = p_2$. Transitivity: within a level the rules encode a lexicographic comparison on position, arity, and key set. For sequential patterns, transitivity is proven by induction on the sequence length $n$: Spec-Tail applies only when both sequences have length $\geq 2$; inductive step by Spec-Prefix when heads are strictly ordered, Spec-Tail when heads satisfy $p_1 \sim q_1$ with the inductive hypothesis on the remainder. If sequences are length 1 with $p_1 \sim q_1$, or if heads differ in form, they are incomparable under $\prec$. For map patterns: Spec-Map-Prefix and Spec-Map-Tail follow the same lexicographic argument over key-sorted position; Spec-Keys follows from transitivity of strict set containment ($\subsetneq$). Across levels, irreflexivity ensures at least one strict level inequality along any $p_1 \prec p_2 \prec p_3$ chain, so Spec-Level or within-level rules give $p_1 \prec p_3$.

*$\prec_{disp}$ is a strict total order.* Incomparability under $\prec$ arises in four cases: (i) patterns differ only in variable names (same form, same level, no positional rule applies); (ii) sequential and map patterns share an arity; (iii) two Level-3 predicates are syntactically distinct with no positional relationship; (iv) two Level-2 patterns specialize on sibling classes. In each case the preference registry and tiebreaker $idx(p)$ supply a unique total rank; since every live clause carries a unique *idx*, no two clauses compare equal under $\prec_{\text{disp}}$. Cases (ii)–(iv) are modularity hazards: without an explicit `prefer-method` declaration, ordering depends on ASDF load sequence. □

Predicate dispatch requires an $O(N)$ scan. FOL re-sorts at definition in $O(ND \log N)$. Subsumption systems achieve $O(1)$ dispatch via expensive provers; FOL trades runtime caching for syntactic tractability.

*4.1.2 Dispatch Examples.* Listing 1 shows a two-clause `defn` transpiled to a `defun` with a static `cond` cascade.

**Listing 1: Transpilation: specificity-ordered multi-clause `defn`**

```
;; FOL source: type specializer (Level 2) before wildcard (Level 0)
(defn classify
  ([x <integer>] :integer)
  ([x]           :other))

;; Generated Common Lisp: ordering preserved as cond cascade
;; (class references bound once via load-time-value)
(defun classify (x)
  (cond
    ((typep x (load-time-value (find-class '<integer>))) :integer)
    (t                                                   :other)))
```

*Comparison to* `defmulti`. Clojure's `defmulti` [16] requires manual resolution via `prefer-method`; FOL's total order eliminates ambiguity for patterns with distinct types or structures.

*4.1.3 Usage Patterns.* Four patterns illustrate these features: structural diff (Listing 2) tracks nested changes by intercepting updates; validated update guard (Listing 3) enforces invariants; predicate dispatch on persistent state (Listing 4) gates behavior based on current values; and content routing (Listing 5) enables value-based alerting. In Listing 2, the `:around` method detects updates and stamps a version counter; the guard `(= key :_changes)` is essential to break the recursion that would otherwise occur when the method itself calls `assoc` to store the change metadata. In Listing 5, the pattern `{:keys [(temp (>= 100))]}` illustrates the power of map-based predicate dispatch: the method only fires if the `:temp` key is present and its value satisfies the predicate, allowing for declarative, behavior-based event routing.

**Listing 2: Automatic Structural Diff (FOL)**

```
(defmethod assoc :around [(obj <diffable>) key val]
  (if (= key :_changes)
      (call-next-method)
        ; key=:_changes: breaks cycle -- calls assoc
        ; directly, skipping the branch that emits
        ; (assoc ... :_changes ...) and re-invokes :around
      (bind [old (get obj key)
             new-obj (call-next-method)]
        (if (= old (get new-obj key))
            new-obj
            (assoc new-obj :_changes (inc (get obj :_changes 0)))))))
```

**Listing 3: Validated Update Guard (FOL)**

```
(defmethod assoc :around [(obj <guarded>) key val]
  (if (valid-update? obj key val)
      (call-next-method)
      obj))  ; discard invalid update; original snapshot unchanged
```

**Listing 4: Predicate Dispatch on Persistent State (FOL)**

```
;; Level-3 predicate (critical?) tests the reading value at dispatch
      time;
;; the body reads persistent slot :alert-history from the persistent
      sensor
```

```
;; object without copying.  Both contributions are required:
      predicate
;; dispatch gates the clause; persistent immutability makes prior
      history
;; a first-class value.
(defmethod record-reading :around
    [(sensor <sensor>) (reading (critical?))]
  (bind [history (get sensor :alert-history [])
         result  (call-next-method)]
    (assoc result :alert-history (conj history reading))))
```

**Listing 5: Value-Based Alerting (FOL)**

```
(defmethod notify [{:keys [(temp (>= 100))]}]
  (print "ALERT: Critical Temperature!"))
```

Predicate forms evaluate their arguments dynamically at each dispatch call, responding to live variable updates (e.g., `*limit*`). However, mutating state inside the predicate closure corrupts dispatch correctness: the dispatch sort (compile-time for `defn`, load-time for `defmethod`) relies on stable predicate structures.

FOL's validated guard avoids the $O(N)$ copy required in CL (Listing 6). While Clojure can achieve per-type guards using multimethods to emulate `:around`, doing so forces all clients of standard `update!` to migrate to the new multimethod; FOL's `:around` integrates securely beneath the existing `assoc` generic function without restructuring the hierarchy.

**Listing 6: Validated Update Guard in Common Lisp — requires explicit copy**

```
(defmethod update-obj :around ((obj guarded) key val)
  ;; No persistent objects: must copy before calling primary to preserve
  ;; original for potential discard.  Cost: O(N) in number of slots.
  (let ((saved (make-instance (class-of obj)
                 :slot-a (slot-value obj 'slot-a)
                 :slot-b (slot-value obj 'slot-b))))
    (if (valid-update-p obj key val)
        (call-next-method)
        saved)))
```

FOL centralizes the guard and discards updates in $O(1)$ via persistence.

(1) The linear scan evaluates Level-3 predicates before type guards; they must be pure and total. (2) Load-order sensitivity affects incomparable patterns (e.g., sibling classes). FOL provides `prefer-method` to resolve these conflicts. (3) Detecting unreachable clauses is future work.

## 4.2 Schema Evolution

Chain replay lazily composes alias maps, recovering renamed values in one pass. Migration confluence prohibits slot names appearing as both source and target, preventing cycles. This is enforced via a DFS within the `reinitialize-instance` MOP protocol.

**Listing 7: Lazy Schema Migration with Aliasing**

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val; :reading keyword still routes to val
(defclass <sensor> [] [[id] [val :alias reading]])
```

At first post-redefinition *mutation*, chain replay writes `:reading`'s value into `:val`; reads of `:reading` before that mutation resolve via the alias map without modifying the instance. Cost is bounded at 5.4–12.2 $\mu$s/instance across 1–3 hops (Section 5.1.2).

# 5 Evaluation

FOL fills a gap between CLOS and Clojure (Table 5), providing MOP-integrated persistence and full CLOS method combinations.

**Table 5: Built-in Feature Comparison**

| Feature | FOL | Clojure | CL |
|---|---|---|---|
| Static type safety | × | × | × |
| Persistent objects | ✓[d] | ○[c] | × |
| CLOS/MOP | ✓ | × | ✓ |
| Method combinations | ✓ | × | ✓ |
| Predicate dispatch | ✓ | single[a] | ×[b] |
| Schema evolution | ✓[e] | × | Manual |
| Transducers | ✓ | ✓ | × |
| Macros | ✓ | ✓ | ✓ |
| Raw scalar performance | Low[f] | High | High |

[a] `defmulti` dispatches via a user-supplied function; `prefer-method`/`derive` allow manual overrides but there is no automatic multi-parameter specificity ordering.
[b] Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.
[c] Clojure has persistent *data structures*; `defrecord` is map-backed, not MOP-integrated.
[d] Slot storage backed by immutable HAMTs; (`setf slot-value`) blocked; updates via `assoc`.
[e] Renamed slots recovered automatically at first post-redefinition access; multi-hop chains composed in a single pass.
[f] Scalar arithmetic and non-slot operations inherit full CL performance. "Low" refers specifically to persistent object slot access: reads add 1.4–2.2× overhead and writes add 40–250× overhead relative to mutable CLOS.

## 5.1 Micro-Benchmarks

We measured persistent objects vs. mutable CLOS across slot counts. **Construction** (≈183 ns, SBCL) is 5× native; `%make-persistent` invokes `allocate-instance` and `initialize-instance` directly, bypassing keyword dispatch while enforcing persistence and stamping `%schema-version`. **Reads** add 1.4–2.2× (native slots); **Updates** are 40–250× slower. Table 7 shows modeled update costs at candidate thresholds. The current implementation uses $T = 32$ (26 user-defined slots plus 6 reserved for the trie pointer and schema metadata): ≈97% of classes (1–26 user-defined slots) incur no overflow, reserving trie allocation only for objects with more than 26 user-defined slots.

**Table 6: Actual slot update overhead ($\mu$s/op) vs. native CLOS**

| Slots | Persist. | Mut. | Ratio | Notes |
|---|---|---|---|---|
| 2 | 1.04 $\mu$s | 24.4 ns | 43× | 2 native slots |
| 8 | 1.32 $\mu$s | 17.2 ns | 77× | 8 native slots |
| 32 | 1.10 $\mu$s | 15.9 ns | 69× | $T = 32$ (native boundary) |
| 48 | 2.16 $\mu$s | 29.1 ns | 74×[a] | 26N + 6S + 16V (overflow update) |
| 128 | 3.78 $\mu$s | 15.2 ns | 249× | 26N + 6S + 96V (overflow update) |

[a] Updating a native slot on a 48-total-slot object costs ≈1.0 $\mu$s (39×); the 74× row benchmarks an overflow-slot update, illustrating the additional trie path-copying cost when the target slot lies beyond the 26 user-defined native positions.

**Table 7: Modeled update cost ($\mu$s/op) at candidate thresholds $T$**

| N | T=8 | T=12 | T=16 | T=24 | T=32 | T=32 strategy |
|---|---|---|---|---|---|---|
| 8 | 0.50 | 0.46 | **0.45** | **0.45** | **0.45** | all-native |
| 12 | 0.80 | 0.67 | 0.66 | **0.65** | 0.66 | all-native |
| 16 | 0.97 | 1.10 | 0.86 | **0.85** | **0.85** | all-native |
| 24 | 1.27 | 1.27 | 1.29 | **1.22** | 1.24 | all-native |
| 32 | **1.52** | 1.56 | 1.55 | 1.69 | 1.64 | all-native |
| 48 | 2.42 | **2.11** | 2.33 | 2.49 | 2.36 | 32N + 16V |

Bold = minimum in row. "$k$N + $j$V" = copy $k$ native slots then batch-build $j$-element overflow vector. All-native = copy $N$ slots directly. Simulated copy costs only; actual `update-slots` additionally incurs MOP iteration overhead (see Table 6).

*5.1.1 Predicate Dispatch Micro-benchmark.* We measured the per-invocation cost of a 20-method generic function, each clause specialized on a distinct Level-3 predicate, comparing FOL open and closed dispatch against standard CLOS type dispatch and a manual `cl:cond` cascade (Table 8, SBCL 2.6.0 x86-64).

**Table 8: Predicate Dispatch Performance ($N = 20$, 5 trials × 1 M iterations).**

| Dispatch Method | Mean/op | Range | Ratio |
|---|---|---|---|
| Standard CLOS (Type) | 31.3 ns | 0.9 ns | 1.00× |
| Manual cond Cascade | 64.0 ns | 2.1 ns | 2.04× |
| FOL `defn` (Closed) | 62.3 ns | 0.4 ns | 1.99× |
| FOL `defgeneric` (Open) | 63.1 ns | 2.1 ns | 2.02× |

FOL `defn` (closed) and `defgeneric` (open) both add ≈2.0× overhead, indistinguishable from the `cl:cond` baseline within noise (±2.1 ns). Open dispatch scans linearly without a cache, so all-Level-2 structures also incur an $O(N)$ penalty. Overhead scales with clause count (Table 9): at $N = 50$ the measured total is 3.6× CLOS.

*5.1.2 Evolution Micro-Benchmark.* Migration costs ≈1.6× CL at 1 hop. At depth, SBCL history machinery dominates; FOL is bounded at ≤ 12.2 $\mu$s and converges with CL at 3 hops (Table 11).

## 5.2 Naive-Translation Workloads

Persistent *slot* updates are 40–250× slower than mutation (Table 6). Naive BFS (Listing 8) illustrates the path-copying penalty (Table 12).

Transients cut allocation significantly and improve BFS overhead from 41.7× to 6.0×.

*Idiomatic Reformulations.* Algorithmic restructuring eliminates the translation penalty (Table 13). **Lazy BFS** initializes nodes on discovery, bypassing $N$ initial `assoc` calls.

Lazy BFS trims initialization waste but its per-edge checks make it slower (18.5×) than naive pre-allocated transients (6.0×).

## 5.3 Abstract Syntax Tree Rewriting Engine

To evaluate both contributions together, we implemented an AST optimization engine: 25 node classes, a single multi-clause `defmethod` encoding 54 algebraic rewrite rules via predicate dispatch (including nested structural patterns such as the identity-element rule below), and a 25-clause `walk` driving recursive descent.

**Listing 8: Naive Persistent BFS Step**

```
(defn bfs-step [q dists graph]
  (if (empty? q)
      dists
      (bind [u (first q)
             edges (get graph u)
             d (get dists u)]
        (loop [es edges nq (rest q) nd dists]
          (if (empty? es)
              (bfs-step nq nd graph)
              (bind [v (first es)]
                (if (= (get nd v) -1)
                    (recur (rest es)
                           (conj nq v)
                           (assoc nd v (inc d)))
                    (recur (rest es) nq nd))))))))
```

```
(defmethod ast-optimize
  [({:keys [(left ({:keys [(val (eql 0))]} <op-lit>))
            (right r)]} <op-add>)]
  r)
```

The built-in `eql` specializer `(val (eql 0))` expands to `(lambda (x) (eql x 0))`, matching a slot against a specific value; it appears in the grammar's predicate position. The CL baseline uses `defstruct` with `typecase` on a 9,999-node balanced tree—`defstruct` is the fastest idiomatic CL baseline, giving the *worst-case* overhead estimate for FOL. Practitioners migrating `defclass`-based code should consult Section 5.5 for the 5.5× adoption-cost estimate under the realistic `defclass`/`defmethod` baseline.

Table 14 shows results. FOL is 202× slower per pass (144× more memory) due to dispatch, reads, and severe GC pressure (35 cycles vs. 1 for CL). The build ratio (116×) exceeds the micro-benchmark because constructing 9,999 nodes runs the full `initialize-instance` MOP protocol per node—stamping versions and initializing overflow tries—with no amortization. Transient Builders (Section 3) mitigate this by allowing mutable construction before freezing into persistent instances.

## 5.4 Logic Simulation (LSim)

LSim is an event-driven digital logic simulator. The **CL** baseline uses a mutable destructive leftist heap [8] with mutable hash-table gate state; **FOL** uses a persistent leftist heap [23] with HAMT gate state. A 2×2 ablation (Table 16) isolates component contributions. Table 15 covers six configurations; the largest circuits used a 56 GB heap.

At scale, the CL baseline allocates one `make-hash-table` per gate (32,768 tables at 32×32×32); as the simulation runs, these objects scatter across the heap and degrade GC scan locality, causing CL's 4.5× per-gate-eval slowdown between the 8×32×32 and 32×32×32 configurations. FOL's HAMT gate-state entries are depth-bounded nursery allocations collected before reaching the old generation; this GC-locality property—not persistence per se—drives the 3.8× advantage. This advantage is specific to the per-gate hash-table allocation pattern: a CL baseline using a single shared hash

**Table 9: Dispatch Scaling: mean time/op (ns) at $N$ clauses (5 trials × 1 M iterations). Ratios vs. CLOS. CLOS is $O(1)$ (type-indexed cache); predicate scan methods are $O(N)$.**

| $N$ | CLOS (ns) | COND (ns) | FOL `defn` (ns) | FOL `defgen` (ns) | COND/× | defn/× | defgen/× |
|---|---|---|---|---|---|---|---|
| 5 | 28.7 | 41.0 | 39.0 | 40.2 | 1.43 | 1.36 | 1.40 |
| 10 | 30.0 | 48.6 | 46.9 | 47.8 | 1.62 | 1.57 | 1.60 |
| 20 | 31.3 | 64.0 | 62.3 | 63.1 | 2.04 | 1.99 | 2.02 |
| 50 | 28.7 | 105.8 | 104.1 | 104.6 | 3.68 | 3.62 | 3.64 |

**Table 10: Single-hop lazy migration: FOL vs. CL ($\mu$s/op, 10K instances)**

| Phase | FOL | CL | Notes |
|---|---|---|---|
| Migration (first touch) | 11.99 | 7.33 | one-time per dormant instance |
| of which: overhead vs. steady-state | 10.62 | 7.31 | SBCL migration + alias recovery |
| Steady-state update | 1.37 | 0.02 | post-migration, repeated use |

**Table 11: Multi-hop lazy migration first-touch cost ($\mu$s/op, 5K instances per cohort)**

| Hop depth | FOL | CL | Notes |
|---|---|---|---|
| 1-hop (v2→v3) | 12.18[b] | 4.10 | FOL allocates new object; CL mutates in-place |
| 2-hop (v1→v3) | 7.59 | 5.62 | FOL's single-pass fold maintains $O(A)$ theoretical scaling. |
| 3-hop (v0→v3) | 5.38 | 5.13 | SBCL migration machinery dominates; FOL and CL costs converge. |
| Steady-state | 1.09 | 0.01 | post-migration; uniform across all cohorts |

[b] Higher than Table 10's 11.99 $\mu$s: instances created at v2 carry two prior class redefinitions in SBCL's version history, incurring more migration overhead than instances created at v1 or v0 regardless of FOL hop depth. This explains the apparent anomaly that 1-hop costs more than 2-hop.

**Table 12: Naive-Translation Workload Results**

| Workload | Implementation | Time | Alloc | vs. CL |
|---|---|---|---|---|
| BFS | CL (hash-table) | 4 ms | 2.3 MB | 1.0× |
| $N$ = 50,000 | FOL Persistent (assoc) | 163 ms | 119 MB | 41.7× |
| | FOL Transient (assoc!) | 24 ms | 13 MB | 6.0× |

**Table 13: Idiomatic Algorithm Results: persistent and transient FOL vs. native CL**

| Workload | Implementation | Time | Alloc | vs. CL |
|---|---|---|---|---|
| BFS lazy | CL (hash-table) | 3 ms | 2.3 MB | 1.0× |
| $N$ = 50,000 | FOL persistent (lazy dict) | 125 ms | 61 MB | 37.8× |
| | FOL transient (`hamt-assoc!`) | 61 ms | 16 MB | 18.5× |

**Table 14: AST Optimizer: FOL vs. CL ($N$ = 100 passes, 9,999-node balanced tree).**

| Phase | Implementation | Time | Alloc | vs. CL |
|---|---|---|---|---|
| Build | CL | 0.516 ms | 0.16 MB | 1.0× |
| | FOL | 60.1 ms | 23.95 MB | 116× |
| Optimizer (per pass) | CL | 0.357 ms/pass | 0.125 MB/pass | 1.0× |
| | FOL | 72.1 ms/pass | 17.91 MB/pass | 202× |

**Table 15: LSim PQ Scaling Results (mean wall time).**

| Config | Gate Evals | CL | FOL | Ratio (FOL/CL) |
|---|---|---|---|---|
| 8bit-100 | 876 | 78 ms | 109 ms | 1.4× |
| 32bit-300 | 10 224 | 94 ms | 141 ms | 1.5× |
| 8x32-900 | 281 248 | 359 ms | 1.87 s | 5.2× |
| 32x32-3000 | 3 850 304 | 7.57 s | 34.15 s | 4.5× |
| 8x32x32-9000 | 89 708 032 | 906 s | 983 s | 1.1× |
| 32x32x32-30000 | 1,183,510,528 | 52,050 s | 13,531 s | 0.26×[a] |

[a] Small-circuit times include shared ≈74 ms initialization overhead; ratio < 1 means FOL is faster. The largest config was ablated (Table 16); the fragmentation explanation is supported by the 2×2 breakdown where persistent maps avoid hash-table degradation.

table keyed by `(gate, slot)` pairs would not exhibit the same fragmentation. A 2×2 ablation (Table 16) breaks down these costs.

**Table 16: 2×2 Ablation: Mutable vs. Persistent Heap × State Maps (mean wall time; ratios relative to CL baseline). Hybrid-A = persistent maps + mutable heap; Hybrid-B = mutable maps + persistent heap; FOL = both persistent.**

| Circuit | CL | Hybrid-A | Hybrid-B | FOL |
|---|---|---|---|---|
| 32bit-300 | 94 ms | 141 ms (1.5×) | 125 ms (1.3×) | 141 ms (1.5×) |
| 8x32-900 | 359 ms | 1.87 s (5.2×) | 2.25 s (6.3×) | 1.87 s (5.2×) |
| 32x32-3000 | 7.57 s | 34.1 s (4.5×) | 42.7 s (5.6×) | 34.1 s (4.5×) |
| 8x32x32-9000 | 906 s | 983 s (1.1×) | 1646 s (1.8×) | 983 s (1.1×) |
| 32x32x32-30000 | 52.0k s | 13.5k s (0.26×) | 70.2k s (1.35×) | 13.5k s (0.26×) |

FOL matches Hybrid-A at 32x32x32-30000: wall-times ranged from 51.8k–52.3k s for CL, and 13.4k–13.6k s for FOL. Differences are within ±110s noise. Hybrid-B falls from 6.3× to 1.35×; as scale grows, the CL baseline itself severely degrades from hash-table fragmentation, making Hybrid-B's fixed $O(\log n)$ heap tax look comparatively better. The 3.8× advantage stems from HAMTs avoiding heap fragmentation.

**Table 17: Expression Interpreter Benchmark (SBCL 2.6.0, AMD Ryzen 9 5900X).**

| Phase | CL | FOL | Time ratio | Mem ratio |
|---|---|---|---|---|
| Build (50 trees, depth 5) | 20.1 ms / 3.8 MB | 35.9 ms / 8.9 MB | 1.8× | 2.3× |
| Eval (1000 iter × 50 exprs) | 0.50 s / 224.5 MB | 2.76 s / 1300 MB | 5.5× | 5.8× |

## 5.5 CLOS Adoption Cost: Expression Interpreter

We port an idiomatic CLOS expression interpreter (seven node types, three generic functions: `eval-expr`, `pretty`, `free-vars`) to FOL with minimal structural changes. Unlike the AST optimizer (Section 5.3), which uses `defstruct` as the fastest-possible CL baseline, this benchmark uses `defclass`/`defmethod`—the natural starting point for a developer adopting FOL from an existing CLOS codebase—giving a more conservative and realistic adoption-cost estimate. The CL baseline uses mutable environments; FOL uses `(assoc env bvar v)`; method bodies are otherwise identical.

A corpus of 50 depth-5 expression trees is evaluated 1,000 times (150,000 root calls across the three generic functions; up to ≈9.4M recursive dispatches).

*Results.* The 5.5× eval ratio reflects a `make-instance` baseline, fewer clauses, and Level-2 specializers using fast `typep` checks. The port requires no architectural logic shifts; differences lie purely in immutable mappings.

# 6 Threats to Validity

**Single implementation**: All measurements use SBCL 2.6.0; results may differ on other CL implementations.

**Experimenter bias**: FOL and all baselines share the same author; more aggressive CL tuning might narrow results.

**Benchmark coverage**: Smaller LSim configs were run repeatedly; the largest circuit was run three times per implementation due to its scale (≈14 hours per run).

**Benchmark selection bias**: Macro-benchmarks may overrepresent FOL's advantages; the interpreter and naive-translation benchmarks offer a more conservative view.

**Interoperability hazard**: CL authors extending FOL classes may encounter runtime mutation errors; these are not signaled at method-definition time.

**Non-portability**: `persistent-class` uses AMOP hooks (`allocate-instance`, `initialize-instance`) but reads SBCL internals. Micro-benchmarks show 1.6× speedup (16.2 vs. 25.7 ns) over portable `slot-value-using-class`.

**Re-evaluation**: interactive re-evaluation reorders incomparable clauses; ordering may differ between sessions if interactive changes occurred.

**All-Level-2**: type-only `defmethod` is expected to incur the same $O(N)$ scan as predicate dispatch, but this is not empirically verified.

# 7 Related Work

*Persistent data structures.* FOL's trie follows Bagwell's HAMT [3] and Clojure [16], using path-copying persistence [10]; Okasaki's heaps [23] underpin our LSim heap. FOL extends libraries like Sycamore [9] and FSet [5] to persistent *objects* with predicate dispatch.

*Schema evolution.* Gemstone [25] and ENCORE [26] pioneered lazy migration, but require explicit coercion code for renames [2]. FOL provides declarative migration via `:alias`; multi-hop chains compose in a single pass via the CLOS MOP. This model is closer in spirit to Datomic's attribute renames, where identity is decoupled from specific attribute keys.

*OOP/FP bridges and pattern dispatch in other languages.* Scala's case classes [22] and `match` [11] provide immutable ADTs with structural dispatch, but updates require explicit `copy` and the class system is not MOP-integrated. Racket [13] separates `match` from its class system [14]; Julia [4] and MultiJava [7] provide multi-parameter type dispatch without predicate specializers.

*Predicate dispatch.* JPred [12] requires a theorem prover for ordering. Millstein [21] achieves decidability by restricting predicates to linear arithmetic; FOL instead admits arbitrary predicates via syntactic leveling, trading decidable subsumption for an open language. Later Cecil work [6] achieves efficient dispatch via offline decision-tree compilation. Our composite level recovers ranking for type-annotated predicates statically. GHC's overlapping instances [15] order heads via pragmas but lack a predicate stratum or evolution. Filtered dispatch [24] introduced guard predicates. Racket's `match` with guards [13] provides structural pattern matching with predicates but is decoupled from its class system. In the Common Lisp ecosystem, libraries like `trivia` offer powerful pattern matching but target `typecase`-like control flow rather than open generic dispatch. Clojure's `defmulti` [16] uses user-supplied functions and `prefer-method`; *protocols* [18] dispatch on the first argument's class. FOL's `defmethod` occupies `defmulti`'s niche but adds predicate specializers and CLOS method combinations, trading an $O(N)$ scan for expressivity.

# 8 Conclusion

FOL contributes a persistent metaclass system and specificity-ordered predicate dispatch. Hybrid storage ($T = 32$) keeps reads fast while lazy migration enables transparent evolution. Specificity ordering handles structurally differentiated patterns, confining load-order sensitivity to structural ties.

Slot updates cost 40–250× slower; FOL is a scale-out solution for complex, versioned architectures rather than a latency-optimized replacement for mutable slots. The dichotomy between the logic simulator's scale advantage and the AST rewrites' 202× penalty provides a concrete heuristic: persistent MOP objects excel for "macro" objects (large state maps, sparse graphs) but are structurally inappropriate for fine-grained pointer structures (like ASTs) due to GC pressure. Future work includes dispatch caching for $O(1)$ amortized cost and static analysis for unreachable clause detection.

FOL is publicly available [1].

## References


[1] Frank Adrian. 2026. *FOL: Functional Object Lisp.* https://github.com/frankadrian314159/fol

[2] Malcolm Atkinson and Ron Morrison. 1995. Orthogonally Persistent Object Systems. *The VLDB Journal* 4, 3 (1995), 319–401.

[3] Phil Bagwell. 2001. *Ideal Hash Trees.* Technical Report 1. EPFL, Lausanne, Switzerland.

[4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.

[5] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. https://common-lisp.net/project/fset/

[6] Craig Chambers and Weimin Chen. 1999. Efficient Multiple and Predicate Dispatching. In *Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 238–255.

[7] Curtis Clifton, Gary T. Leavens, Craig Chambers, and Todd Millstein. 2000. MultiJava: Modular Open Classes and Symmetric Multiple Dispatch for Java. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 130–145.

[8] Clark Allen Crane. 1972. *Linear Lists and Priority Queues as Balanced Binary Trees.* Technical Report STAN-CS-72-259. Stanford University.

[9] Neil T. Dantam. 2013. Sycamore: Purely Functional Data Structures for Common Lisp. https://github.com/ndantam/sycamore

[10] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.

[11] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007–Object-Oriented Programming*. Springer, Berlin, Heidelberg, 273–298.

[12] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP'98–Object-Oriented Programming*. Springer, Berlin, Heidelberg, 186–211.

[13] Matthew Flatt et al. 2015. The Racket Manifesto. In *20 Years of the Past and 20 Years of the Future (Snapshots), Dagstuhl Seminar 15451.* https://doi.org/10.4230/DagRep.5.11.112

[14] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.

[15] GHC Team. 2004. *GHC User's Guide: Overlapping Instances.* Glasgow Haskell Compiler. https://downloads.haskell.org/ghc/latest/docs/users_guide/.

[16] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. ACM, New York, NY, USA, 1–10.

[17] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md

[18] Rich Hickey. 2010. Clojure Protocols. Clojure Reference Documentation. https://clojure.org/reference/protocols

[19] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol.* MIT press.

[20] Michele Lanza and Radu Marinescu. 2006. *Object-Oriented Metrics in Practice.* Springer.

[21] Todd Millstein. 2004. Practical Predicate Dispatch. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 345–364.

[22] Martin Odersky, Lex Spoon, and Bill Venners. 2016. *Programming in Scala, 3rd Edition.* Artima Press.

[23] Chris Okasaki. 1998. *Purely Functional Data Structures.* Cambridge University Press.

[24] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. ACM, New York, NY, USA, 20–26.

[25] D. J. Penney and J. Stein. 1987. Class Modification in the GemStone Object-Oriented DBMS. In *Proceedings of the 2nd ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 111–117.

[26] Andrea H. Skarra and Stanley B. Zdonik. 1986. The Management of Changing Types in an Object-Oriented Database. In *Proceedings of the 1st ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 483–495.

[27] Philip Wadler. 1990. Linear Types Can Change the World!. In *Programming Concepts and Methods*. North-Holland, Amsterdam, Netherlands, 347–359.